AI Answer Sheet Evaluator

Technical Documentation and Teacher Presentation Guide

A local-first system for evaluating handwritten, scanned, and digital answer sheets against expected answers and teacher marking points.

Project field	Value
Version	3.0.0
Application type	Local Flask web application
Primary goal	OCR-tolerant automated answer evaluation
Primary input	PDF and image answer sheets
Primary output	Scores, feedback, history, and PDF reports
Documentation date	2026-06-10
Repository	https://github.com/gourav34135/Ai_answersheet_evaulation_system

1. Executive Summary

The AI Answer Sheet Evaluator is designed to reduce the time required to review descriptive answers while keeping a teacher in control of the expected answer and marking rubric. The user can upload any supported answer-sheet file, enter any question and expected answer, and receive an overall score plus per-question evidence.

The central technical challenge is that handwriting OCR is imperfect. A correct answer may be transcribed with spelling errors. The project addresses this by combining image preprocessing, OCR, reference-assisted cleanup, fuzzy concept matching, semantic similarity, character n-gram similarity, rubric coverage, and completeness analysis.

2. Project Objectives

• Accept arbitrary PDF and image answer sheets rather than a fixed sample.

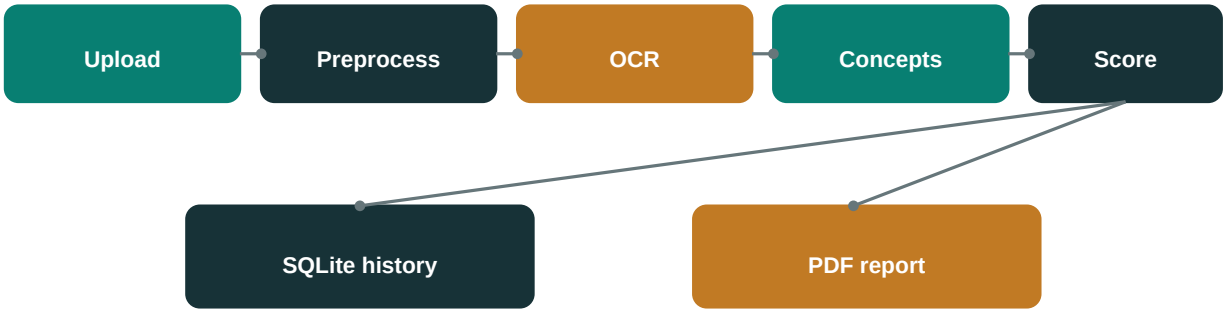
• Allow teachers or students to supply their own questions, expected answers, marking points, and maximum score.

• Provide fair credit when OCR contains character-level mistakes but the correct concepts remain detectable.

• Generate per-question results and a downloadable evidence-based PDF report.

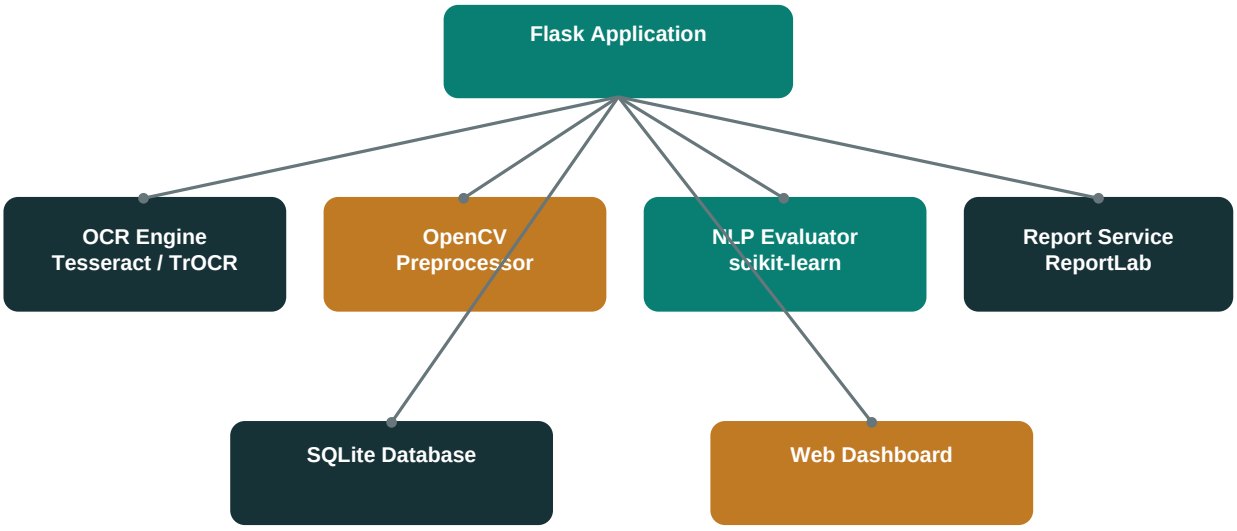
• Keep uploaded files and history local for privacy and offline operation.

3. End-to-End Workflow



The uploaded document is validated, rendered when necessary, preprocessed, transcribed, cleaned using expected-answer vocabulary, evaluated using multiple NLP signals, saved locally, and converted into a downloadable report.

4. Software Architecture



Component	Responsibility
Flask application	Routes, upload validation, evaluation orchestration, history, and downloads
OCR engine	Digital PDF extraction, Tesseract OCR, and optional TrOCR recognition
Image preprocessor	Upscaling, ink isolation, denoising, thresholding, deskewing, and line cleanup
NLP evaluator	Semantic, fuzzy, concept, rubric, completeness, and per-question scoring
SQLite database	Local persistence of results, feedback, and OCR text
Report service	Professional PDF evaluation report generation
Web dashboard	Interactive input, progress, result analysis, and history review

5. Technology Stack

Technology	Use in project
Python	Core application language and evaluation implementation
Flask / Werkzeug	Local web server, routes, APIs, uploads, and downloads
Tesseract / pytesseract	Default offline OCR engine
Microsoft TrOCR	Optional transformer model for handwritten single-line recognition
OpenCV / Pillow / NumPy	Document-image preprocessing
PyMuPDF	PDF rendering and digital-text extraction
scikit-learn	Word and character TF-IDF cosine similarity
difflib	OCR-tolerant fuzzy token matching
SQLite	Local evaluation history
ReportLab	Evaluation and technical PDF generation

Technology	Use in project
HTML / CSS / JavaScript	Interactive dashboard
unittest	Regression verification

6. OCR Models and Image Processing

Standard OCR

The dependable offline path uses Tesseract. Each scanned PDF page is rendered by PyMuPDF and preprocessed before OCR. The system tries document-appropriate page segmentation modes and selects the strongest transcription candidate.

Optional Transformer OCR

The advanced path uses Microsoft's trocr-base-handwritten model through PyTorch and Hugging Face Transformers. TrOCR is a pretrained vision-encoder-decoder model fine-tuned on IAM handwritten text and is intended for single text-line images. The system segments candidate lines before inference.

A model was not trained from scratch because reliable handwriting-model training requires a large labeled dataset containing line images and exact transcriptions, substantial GPU compute, and careful evaluation across different writers. Instead, the project uses pretrained OCR models and calibrates the downstream evaluator with noisy-OCR regression tests. This is more reproducible for a local academic project.

Preprocessing Operations

- Border trimming and image upscaling.
- Blue and purple ink isolation for ruled-paper answers.
- Grayscale conversion, contrast enhancement, and sharpening.
- Adaptive thresholding for uneven lighting.
- Noise reduction and ruled-line removal.
- Deskewing and line-region segmentation.

7. Evaluation Algorithms

Signal	Technical method	Purpose
Semantic similarity	TF-IDF word n-grams and cosine similarity	Measures meaning and vocabulary overlap
Character similarity	TF-IDF character 3-5 grams	Tolerates OCR spelling corruption
Concept coverage	Fuzzy expected-term matching per answer section	Detects required subject concepts
Rubric coverage	Exact, fuzzy, and generic rubric rules	Checks marking-point satisfaction
Completeness	Student/reference word-length comparison	Detects incomplete responses
Readability	Vocabulary diversity and sentence structure	Adds a small presentation-quality signal
Question scoring	Numbered reference-section evaluation	Produces per-question evidence

The final score is a weighted combination of semantic similarity, concept coverage, rubric coverage, completeness, readability, and aggregated question results. The weights are configurable in evaluator.py.

8. Data, Privacy, and Security

• Uploaded answer sheets are stored only inside the local uploads directory.

• Evaluation history is stored in a local SQLite database.

• No answer sheet is sent to an external API by the standard OCR path.

• File extensions and upload size are validated before processing.

• The optional TrOCR model download contacts the model repository only during model installation or first use.

• Teachers should review generated scores before using them as final academic grades.

9. Main API Endpoints

Method	Endpoint	Function
POST	/api/evaluate	OCR and evaluate an uploaded answer sheet
POST	/api/rescore	Evaluate supplied text
GET	/api/history	List saved results
DELETE	/api/history	Clear local result history
GET	/api/history/<id>	Load one saved evaluation
GET	/api/report/<id>	Download the final PDF report
GET	/api/health	Check application version and Tesseract availability

10. Testing and Validation

The project includes an automated noisy-OCR regression test and was validated using both a multi-page handwritten machine-learning answer sheet and an unrelated digital science answer sheet. The purpose of this validation is to confirm that evaluation is based on user-provided expected answers rather than hard-coded sample content.

Validation	Observed result
Noisy OCR regression test	Passed
Handwritten multi-page PDF	Five per-question results and complete rubric analysis
Unrelated science PDF	Two per-question results based on separate science references
PDF evaluation report	Generated and visually inspected
Browser dashboard	Desktop and responsive result surfaces verified

11. Installation and Execution

macOS Standard Setup

```
brew install tesseract
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

```
python app.py
```

```
Open http://127.0.0.1:5000
```

Optional Transformer Setup

```
python3.12 -m venv .venv-transformer
```

```
source .venv-transformer/bin/activate
```

```
pip install -r requirements-advanced.txt
```

12. Limitations

- No OCR model can guarantee perfect transcription of every handwriting style.
- Heavy shadows, curved pages, low resolution, and overlapping writing reduce OCR quality.
- Reference answers and marking points must be academically correct and sufficiently detailed.
- The current evaluator grades concepts but does not independently verify every factual claim.
- Transformer OCR requires additional memory, disk space, and model-download time.

13. Future Scope

- Fine-tune TrOCR on institution-specific handwritten answer-sheet transcriptions.
- Add teacher review and score-adjustment workflows.
- Add separate answer-region detection for structured exam templates.
- Add multilingual OCR and subject-specific scoring profiles.
- Add authentication and class-level analytics for multi-user deployment.
- Add calibrated factual-consistency checking with a locally hosted language model.

14. Viva Summary

This project is not simply an OCR reader. It is an OCR-tolerant evaluation system. Computer vision prepares the document, OCR transcribes it, NLP compares meaning and concepts, fuzzy matching reduces the effect of OCR spelling errors, SQLite stores local evidence, and ReportLab generates a professional report. The teacher remains responsible for the final academic decision.